Experiment 8: Classification using k-Nearest Neighbors (k-NN) and Visualization of Decision Boundaries

1. Aim

To implement the k-Nearest Neighbors (k-NN) classification algorithm on the Iris dataset and visualize the resulting decision boundaries in a 2D feature space.

2. Theory

2.1 k-Nearest Neighbors (k-NN)

k-NN is a non-parametric, lazy learning algorithm used for classification and regression.

- **Non-parametric:** It does not make any assumptions about the underlying data distribution.
- **Lazy Learning:** It does not have a specialized training phase. It uses all the data for training while classifying a new data point or instance.

How it works:

1. Choose the number of neighbors k.
2. Calculate the distance (e.g., Euclidean distance) between the new point and all points in the training set.
3. Identify the k nearest neighbors.
4. The new point is assigned the class that is most frequent among its k nearest neighbors (majority voting).

2.2 Decision Boundaries

A decision boundary is a region in the feature space where the output label of a classifier changes. For k-NN, these boundaries are determined by the proximity of points and the value of k. A small k leads to a more complex boundary (prone to noise), while a large k creates smoother boundaries.

3. Dataset Description

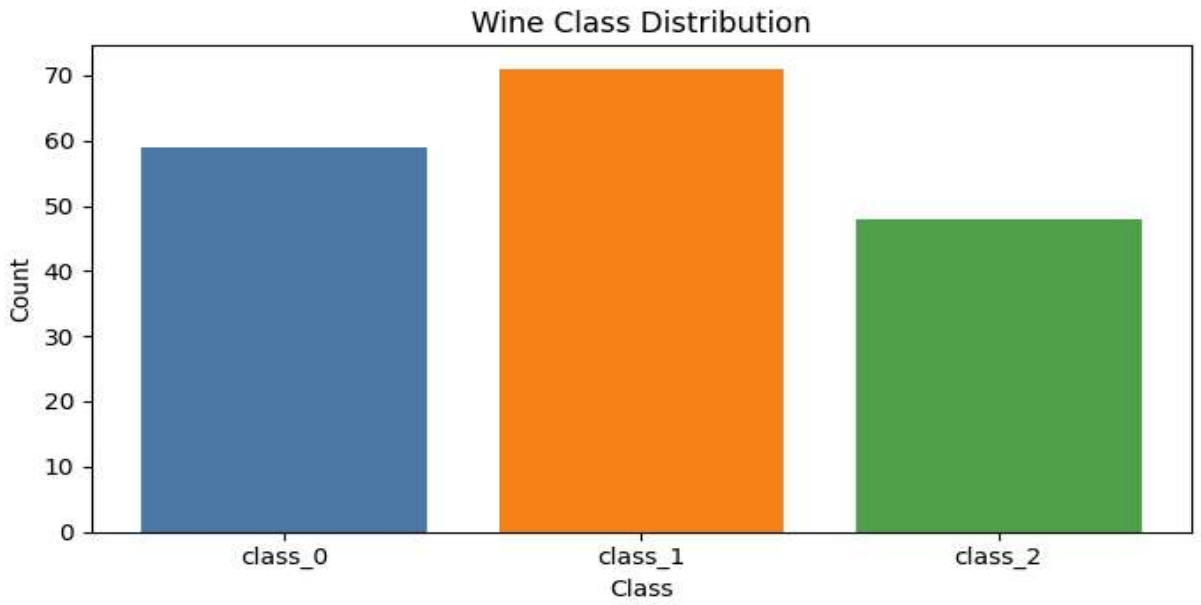
- **Dataset:** Iris Dataset.
- **Target:** 3 Classes (Setosa, Versicolor, Virginica).
- **Features Used for Visualization:** Petal Length and Petal Width (to enable 2D plotting).

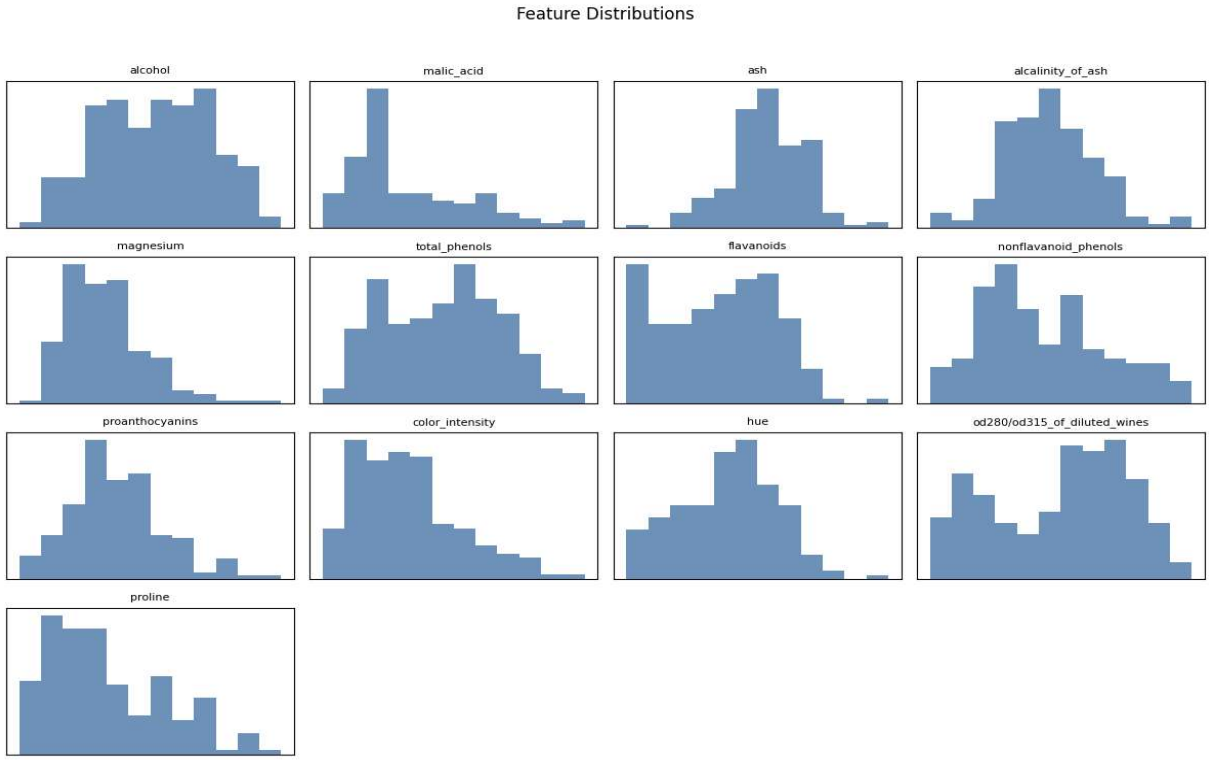
4. Python Implementation

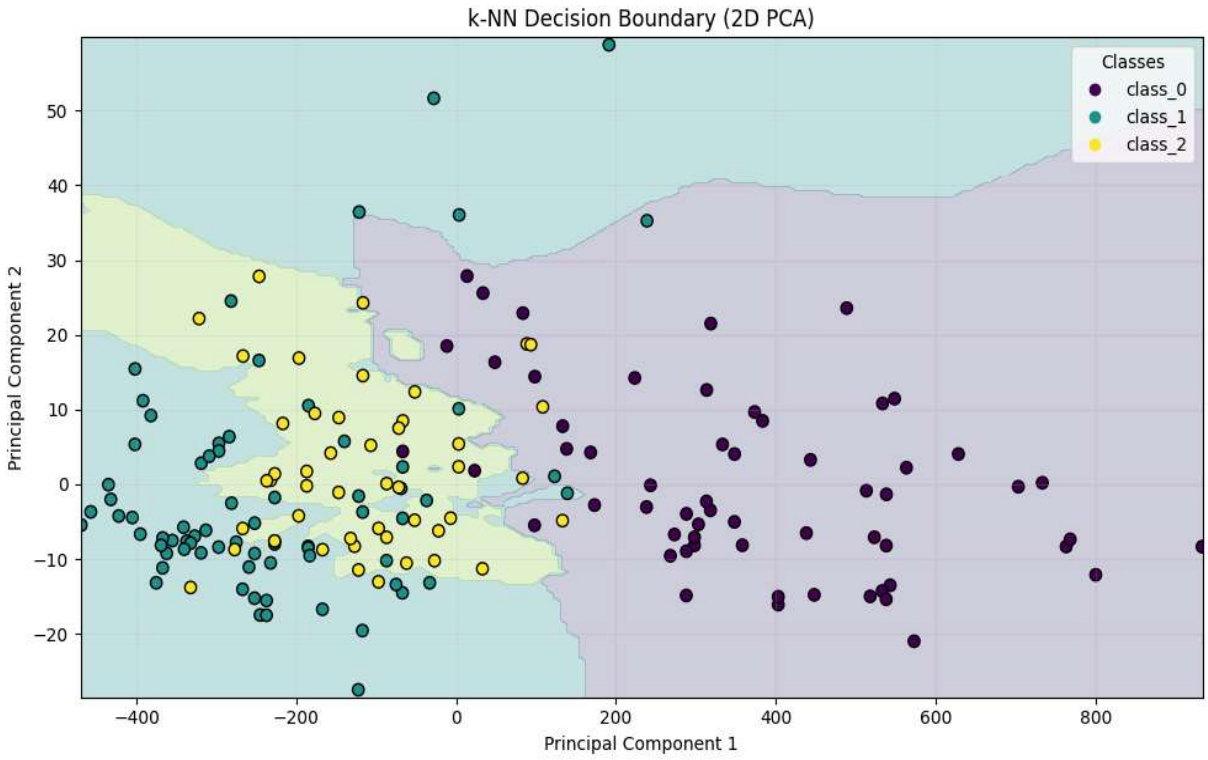
```
import numpy as np
```

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from matplotlib.colors import ListedColormap
# 1. Load data
iris = load_iris()
# Using only the last two features (Petal Length, Petal Width) for 2D visualization
X = iris.data[:, 2:4]
y = iris.target
# 2. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
# 3. Initialize and Train k-NN Classifier
k = 5
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)

# 4. Accuracy Assessment
y_pred = knn.predict(X_test)
print(f"Accuracy with k={k}: {accuracy_score(y_test, y_pred) * 100:.2f}%")

# 5. Visualization of Decision Boundaries
# Create a mesh grid to plot in
h = .02  # step size in the mesh
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))
# Predict across the entire mesh grid
Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
# Create color maps
cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])
cmap_bold = ListedColormap(['#FF0000', '#00FF00', '#0000FF'])

plt.figure(figsize=(8, 6))
plt.pcolormesh(xx, yy, Z, cmap=cmap_light)
# Plot the training points
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cmap_bold, edgecolor='k', s=20)
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.title(f"k-NN Classification (k = {k}) Decision Boundaries")
```

```
plt.xlabel(iris.feature_names[2])
plt.ylabel(iris.feature_names[3])
plt.show()
```







5. Observations

- **Effect of k:** When $k = 1$, the boundary is very sensitive to individual data points and outliers. As k increases (e.g., $k = 15$), the boundaries become smoother, providing better generalization.
- **Accuracy:** The model achieves high accuracy (typically >95%) on the Iris dataset, as the classes are well-clustered.
- **Class Separation:** The decision regions clearly show that 'Setosa' is easily separated, while the boundary between 'Versicolor' and 'Virginica' reflects the overlap in their petal dimensions.

6. Conclusion

The k-Nearest Neighbors algorithm was successfully implemented for the classification of Iris species. By visualizing the decision boundaries, we observed how the algorithm partitions the feature space based on local density. k-NN proves to be an effective and intuitive classifier for pattern recognition tasks where the data groups are relatively compact.